

ESI

Informe Servidor Grafana grupo 02



Universidad de Valladolid

Alfredo del Val Ramos

Daniel Garcia Salinas

Francisco Ivan San Segundo Alvarez

Gabriel Ferrero Herrera

31 de marzo de 2025

Índice general

1. Correccion.....	3
2. Scripts.....	3
2.1. Conexion a la base de datos e insercion de los mismos	3
2.2. Borrado de datos en intervalo	4
2.3. Repetición del script.....	4
2.4. Carga instantanea de CPU	5
2.5. Frecuencia instantanea de CPU	6
2.6. Uso de memoria RAM y memoria SWAP	7
2.7. Espacio de disco	8
2.8. Operaciones de E/S	9
2.9. Estadisticas de red. Bytes enviados y recibidos	11
2.10. Conexiones de red activas	11
2.11. Numero de procesos en ejecucion.....	12
2.12. Lista detallada de procesos en ejecucion.....	13
2.13. Hilos	14
2.14. Inodos.....	15
2.15. Interrupciones y forks	16
2.16. Sesiones	16
3. Base de datos	19
3.1. info_carga_cpu.....	19
3.2. frecuencia_cpu	19
3.3. uso_ram_swap	20
3.4. espacio_disco	20
3.5. operaciones_es	20

3.6. bytes_red.....	21
3.7. conexiones_red	21
3.8. nprocesos_ejecuciones.....	22
3.9. info_procesos	22
4. Graficas y Valores	24
4.1. ESPACIO USADO Y LIBRE EN DISCO	24
4.2. USO DE RAM Y SWAP	25
4.3. DIFERENCIA DE INODOS DEL DISCO DURO.....	26
4.4. USO DE CPU Y MEMORIA POR PARTE DE LOS HILOS	277
4.5. NUMERO DE PROCESOS	29
4.6. INTERCONEXION DE PROCESOS	30
4.7. INFORMACION DE PROCESOS	32
4.8. INTERRUPCIONES Y FORKS.....	33
4.9. OPERACIONES ENTRADAS/SALIDAS.....	35
4.10. SESIONES	36
4.11. FRECUENCIA CPU	37
4.12. CARGA CPU	38
4.13. BYTES ENVIADOS Y RECIBIDOS	39
4.14. INFORMACION DE CONEXIONES DE RED	41
5. Conclusiones	42

1. Correccion

IMPORTANTE: Para la correccion facilitamos el siguiente usuario de Grafana

Usuario: correccion

Contraseña: correccionESI

2. Scripts

En esta seccion se van a detallar todos los scripts usados para sacar los valores de rendimiento del servidor, perfectamente documentados. Los scripts se han realizado en el lenguaje *"shellscript"*.

2.1. Conexion a la base de datos e insercion de los mismos

Todos los scripts deben insertar los datos recogidos en la base de datos, por lo que para no ser redundantes en las siguientes secciones se expone en esta la estructura general que siguen los scripts

```
#!/bin/bash

#Script para la obtención del espacio en disco

DB_HOST="localhost"
DB_USER="root"
DB_PASS="someselgrupo02*"
DB_PATH="esi_proyecto"

#Obtenemos el timestamp
timestamp=$(date +%Y-%m-%d %H:%M:%S)

mysql -h "$DB_HOST" -u "$DB_USER" -p"$DB_PASS" "$DB_PATH" -e "INSERT INTO espacio_disco VALUES ($datos, '$timestamp');"
```

Figura 2.1: Conexion a la base de datos e insercion de los mismos

Como podemos apreciar en la imagen, encontramos 4 variables con valores asignados para la conexion de la base de datos: el host a conectarse, el usuario, la contraseña y el path en el que

estamos trabajando. Tenemos otra variable presente, el timestamp, que usamos para correlacionar los datos con la fecha y la hora a la que se tomaron.

En la ultima linea del script observamos para que sirven todos estos datos, pues nos conectamos a la base de datos con las primeras 4 variables, posteriormente escribimos el comando para anadir los valores a la tabla que corresponde y por ultimo adjuntamos las variables de los valores junto con el timestamp.

Esta estructura va a estar presente en todos los cripts usados en el proyecto, por lo que es conveniente redactarla individualmente para entenderla mejor.

2.2. Borrado de datos en intervalo

En ciertos scripts, para disminuir la cantidad de datos en la base de datos y que no sobrecarguen el sistema, se usa la sentencia SQL de la foto para disminuir por intervalos el numero de entradas en la tabla:

```
#Elimino los datos que tengan mas de un dia de antigüedad
mysql -h "$DB_HOST" -u "$DB_USER" -p"$DB_PASS" "$DB_PATH" -e "DELETE FROM interrupciones_forks where col_timestamp < NOW
() - INTERVAL 1 DAY;"
```

Figura 2.2: Conexion a la base de datos e insercion de los mismos

2.3. Repetición del script

En nuestro caso, hemos decidido configurar los scripts como servicios o "daemons". Cuando el sistema se inicia, gracias a los ficheros en /etc/systemd/system", los scripts empezarian a funcionar:

```
root@virtual:/etc/systemd/system# ls | grep grafana.service
bytes-enviados-recibidos_grafana.service
carga_grafana.service
conexiones_red_activas_grafana.service
espacio_disco_grafana.service
frecuencia_grafana.service
hilos_grafana.service
info_procesos_grafana.service
inodos_grafana.service
interrupts-forks_grafana.service
numprocesos_grafana.service
operacionesentradas_grafana.service
sesiones_grafana.service
uso_ram_swap_grafana.service
```

Figura 2.3: Conexion a la base de datos e insercion de los mismos

2.4. Carga instantanea de CPU

El script se centra en la tarea de monitorear el uso de la CPU. Para lograr esto, utilizamos el comando `"ps -eo%cpu -no -headers"` junto con `awk '{s+=$1} END {print s}'`. Este conjunto de comandos nos da el valor que representaba la carga total de CPU de todos los procesos.

Para hacer real el valor lo ajustamos relacionando la carga total con el numero de nucleos disponibles, obteniendo este numero utilizando el comando `"nproc"`. Este valor nos indica cuantas unidades de procesamiento estan disponibles. Luego, se calcula un valor de uso de CPU normalizado, dividiendo la suma total de la carga de procesos entre el numero de nucleos.

```

while true;do
# Sumar directamente el %CPU de todos los procesos
cpu_total=$(ps -eo %cpu --no-headers | awk '{s+=$1} END {print s}')
# Verificar si la variable está vacía o NULL
if [[ -z "$cpu_total" ]]; then
    echo "Error: No se pudo obtener el uso de CPU correctamente."
    exit 1
fi
# Obtener el número de núcleos lógicos
cores=$(nproc)
# Calcular el % normalizado
cpu_normalized=$(echo "$cpu_total / $cores" | bc -l)
# Insertar el valor normalizado
mysql -h "$DB_HOST" -u "$DB_USER" -p"$DB_PASS" "$DB_PATH" -e "INSERT INTO info_carga_cpu (carga_instantanea) VALUES ($cpu_normalized);",
sleep 60
done

```

Figura 2.4: Carga instantanea de CPU

2.5. Frecuencia instantanea de CPU

Para obtener la frecuencia instantanea usamos el archivo `/proc/cpuinfo`. Para ello, extraemos la frecuencia en MHz del primer nucleo del procesador mediante un pipeline de comandos: `grep` busca la linea especifica con `cpuMHz`, `awk` extrae 1 valor numerico y `head` asegura que solo se tome 1 dato del primer nucleo.

El valor obtenido en MHz se convierte a GHz dividiendolo entre 1000, utilizando 1 comando `bc` para precision decimal. Para evitar errores en sistemas con configuraciones regionales que usan comas como separadores decimales, se fuerza 1 formato numerico con `LC_NUMERIC=C`, garantizando que 1 calculo (por ejemplo, convertir 3200 MHz a 3.20 GHz) se interprete correctamente.

```

while true;do
# Obtener frecuencia y convertir a GHz (¡usa LC_NUMERIC=C!)
cpu_freq=$(grep "cpu MHz" /proc/cpuinfo | awk '{print $4}' | head -n1)
cpu_freq_ghz=$(LC_NUMERIC=C echo "scale=2; $cpu_freq / 1000" | bc)
# Insertar en la base de datos (¡con comillas alrededor del valor!)
mysql -h "$DB_HOST" -u "$DB_USER" -p"$DB_PASS" "$DB_PATH" -e \
    "INSERT INTO frecuencia_cpu (velocidad) VALUES ({cpu_freq_ghz});",
sleep 60
done

```

Figura 2.5: Frecuencia instantanea de CPU

2.6. Uso de memoria RAM y memoria SWAP

Lo que hacemos en este script es leer cada línea que nos sale del comando *"free-h"*, el cual nos sirve para mostrar el uso de la memoria del sistema de manera legible, utilizando el flag *"-h"* para convertir los valores de memoria a una unidad manejable (MB o GB, en lugar de bytes).

Dentro del bucle, tenemos la variable *"tipo"* donde guardamos el nombre de la memoria cuya información estamos leyendo. En la variable *"usada"* guardamos la cantidad de la memoria que se está usando, utilizando el comando *"sed"* que nos permite quedarnos exclusivamente con los números por un lado y la unidad de los mismos por otro, guardándolo en la variable *unidad*.

Cuando ya tenemos ambos datos, utilizamos un *case* para reemplazar la coma por el punto de manera que cuando lo insertemos en la base de datos podamos guardarlo en un *float* y convertimos los datos a GB. Con la cantidad de memoria libre hacemos lo mismo. Una vez que ya tenemos la cantidad de memoria usada y la cantidad de memoria libre, si el tipo de memoria que hemos leído es la memoria RAM metemos los datos en *"usada_ram"* y *"libre_ram"*, si, por el contrario, el tipo de memoria que hemos leído es la memoria Swap, guardamos los datos en la variable *"usada_swap"* y *"libre_swap"*.

Leemos la siguiente línea y realizamos el mismo proceso, sin embargo, ahora tratándose del otro tipo de memoria.

```

usado_ram=
usado_swap=
libre_ram=
libre_swap=

#Elimino los datos que tengan mas de un día de antigüedad
mysql -h "$DB_HOST" -u "$DB_USER" -p"$DB_PASS" "$DB_PATH" -e "DELETE FROM uso_ram_swap where cdf_timestamp < NOW() - (INTERVAL 1 DAY)"

while read -r linea;do
    tipo=$(echo "$linea" | awk '{print $1}')
    usado=$(echo "$linea" | awk '{print $2}' | sed 's/[a-zA-Z]//g')
    unidad=$(echo "$linea" | awk '{print $3}' | sed 's/[a-zA-Z]//g')
    case $unidad in
        G|)
            usado_limpio=$(echo "$usado" | awk '{print $1/1024}' | sed 's/././g')
            ;;
        K|)
            usado_limpio=$(echo "$usado" | awk '{print $1/1024}' | sed 's/././g')
            ;;
        M|)
            usado_limpio=$(echo "$usado" | sed 's/././g')
            ;;
        T|)
            usado_limpio=$(echo "$usado" | awk '{print $1/1024*1024}' | sed 's/././g')
            ;;
    esac

    libre=$(echo "$linea" | awk '{print $4}' | sed 's/[a-zA-Z]//g')
    unidad=$(echo "$linea" | awk '{print $5}' | sed 's/[a-zA-Z]//g')
    case $unidad in
        G|)
            libre_limpio=$(echo "$libre" | awk '{print $1/1024}' | sed 's/././g')
            ;;
        K|)
            libre_limpio=$(echo "$libre" | awk '{print $1/1024}' | sed 's/././g')
            ;;
        M|)
            libre_limpio=$(echo "$libre" | sed 's/././g')
            ;;
        T|)
            libre_limpio=$(echo "$libre" | awk '{print $1/1024*1024}' | sed 's/././g')
            ;;
    esac

    if [ "$tipo" = "mem" ]; then
        usado_ram=$(echo "$usado_limpio")
        libre_ram=$(echo "$libre_limpio")
    else
        usado_swap=$(echo "$usado_limpio")
        libre_swap=$(echo "$libre_limpio")
    fi
done < <(free -h | tail -n 4)

mysql -h "$DB_HOST" -u "$DB_USER" -p"$DB_PASS" "$DB_PATH" -e "INSERT INTO uso_ram_swap VALUES ($usado_ram,$libre_ram,$usado_swap,$libre_swap,$timestamp);"

sleep 60
done

```

Figura 2.6: Uso de memoria RAM y memoria SWAP

2.7. Espacio de disco

Lo primero que hacemos es borrar los datos que haya en la base de datos para actualizar el grafico y que solo aparezcan los ultimos datos que vamos a recoger.

Vamos leyendo cada linea que devuelve el comando "*df -h/dev/sda2*", el cual nos sirve para mostrar el espacio en disco de la particion "*/dev/sda2*" en un formato comprensible (con unidades como GB, MB ,etc..).

Con cada linea que leemos del comando, guardamos en la variable "*usado*" la cantidad de espacio de disco que esta usado por esta particion y la unidad en la que se muestra esta informacion. Despues utilizamos un case para reemplazar el formato de datos con coma a formato de dato con punto haciendo de esta manera que sea despues insertable en un formato de tipo float y convertirlo a MB. Una vez que tenemos los datos anteriormente comentados convertidos y en el formato correcto, hacemos lo mismo con el espacio de memoria libre de esta misma particion.

Al igual que hicimos con el espacio de disco, convertimos la cantidad de espacio libre al formato punto y a MB. A continuacion, obtenemos el nombre de la particion de la cual estamos mirando el espacio libre y usado y sumamos una unidad al id, esta variable sirve como clave primaria de la base de datos.

Una vez que ya tenemos todos los datos, los insertamos en la base de datos y repetimos este proceso mientras que haya lineas del comando que no hayamos leído.

```

Timestamp=$(date +%Y-%m-%d_%H-%M-%S)
sid=0
#Elimino los datos que existan en la tabla para introducir los nuevos
mysql -h "$DB_HOST" -u "$DB_USER" -p"$DB_PASS" "$DB_PATH" -e "DELETE FROM espacio_disco;"
#Obtenemos el espacio en disco
df -h /dev/sda2 >>/dev/null | tail -n1 | while read linea; do
    usadon=$(echo "$linea" | awk '{print $3}' | sed 's/[A-Za-z]//g')
    unidon=$(echo "$linea" | awk '{print $4}' | sed 's/[A-Za-z]//g')
    case $usadon in
        G)
            usado_limpio=$(echo "$usadon" | sed 's/././')
            ;;
        M)
            usado_limpio=$(echo "$usadon" | awk '{print $1/10240}' | sed 's/././')
            ;;
        T)
            usado_limpio=$(echo "$usadon" | awk '{print $1/1024}' | sed 's/././')
            ;;
        K)
            usado_limpio=$(echo "$usadon" | awk '{print $1/1000000}' | sed 's/././')
            ;;
    esac
    libre=$(echo "$linea" | awk '{print $5}' | sed 's/[A-Za-z]//g')
    unidol=$(echo "$linea" | awk '{print $6}' | sed 's/[A-Za-z]//g')
    case $libre in
        G)
            libre_limpio=$(echo "$libre" | sed 's/././')
            ;;
        M)
            libre_limpio=$(echo "$libre" | awk '{print $1/1024}' | sed 's/././')
            ;;
        K)
            libre_limpio=$(echo "$libre" | awk '{print $1/1000000}' | sed 's/././')
            ;;
        T)
            libre_limpio=$(echo "$libre" | awk '{print $1/1024}' | sed 's/././')
            ;;
    esac
    numero=$(echo "$linea" | awk '{print $1}')
    id=$((C+140))
    mysql -h "$DB_HOST" -u "$DB_USER" -p"$DB_PASS" "$DB_PATH" -e "INSERT INTO espacio_disco VALUES ($usado_limpio,$libre_limpio,$nombre,$id,$Timestamp);"
done
sleep 10
done

```

Figura 2.7: Espacio de disco

2.8. Operaciones de E/S

Este script se encarga de obtener y registrar las operaciones de entrada/salida (E/S) del sistema, utilizando la informacion del archivo *"/proc/diskstats"*.

A traves del uso de *"awk"*, el script extrae las estadisticas de E/S del archivo *"/proc/diskstats"*.

Se calculan tres valores principales:

1. -Operaciones totales de E/S: Calculadas sumando las lecturas y escrituras, multiplicadas por 2, ya que lo que nos ofrece *"/proc/diskstats"* son las lecturas de bloques de disco y para pasarlo a KB necesitamos multiplicarlo por 2 ($512 * 2 = 1024$).
2. -Operaciones de lectura: Calculadas a partir del cuarto campo del archivo *"/proc/diskstats"*.
3. -Operaciones de escritura: Calculadas a partir del octavo campo del archivo *"/proc/diskstats"*.

Calculo de diferencias con la ultima ejecucion: El script consulta la base de datos, mas exactamente en la tabla *"operaciones_es"*, para obtener los ultimos valores registrados de las operaciones de lectura, escritura y el total de operaciones. A partir de estos valores, calcula las diferencias con respecto a la ejecucion anterior, para asi poder obtener los nuevos valores de diferencia en el tiempo establecido y no unicamente el valor acumulado.

```
while true;do
#Obtenemos la fecha actual
timestamp=$(date +%s-%m-%d-%H-%M-%S)

#Filtra los datos que tengan mas de un dia de antigüedad
mysql -h "$DB_HOST" -u "$DB_USER" -p"$DB_PASS" "$DB_PATH" -e "DELETE FROM operaciones_es where ts_timestamp < NOW() - INTERVAL 1 DAY;"

#Obtenemos el numero de operaciones de E/S, desde el fichero /proc/diskstats y vamos acumulandolas en una
operaciones_es=$(cat /dev/null > /dev/null & { echo $(cat /proc/diskstats) | awk '{sum+=$4+$8}' } & | tail -n 1)
lecturas_es=$(cat /dev/null > /dev/null & { echo $(cat /proc/diskstats) | awk '{sum+=$4}' } & | tail -n 1)
escrituras_es=$(cat /dev/null > /dev/null & { echo $(cat /proc/diskstats) | awk '{sum+=$8}' } & | tail -n 1)

last_id=$(mysql -h "$DB_HOST" -u "$DB_USER" -p"$DB_PASS" "$DB_PATH" -e "SELECT id_ultima FROM operaciones_es;" -B | tail -n 1)
if [ -z "$last_id" ]; then
    id=0
    min_read=0
    min_write=0
    operaciones_tot=0
else
    min_read=$(mysql -h "$DB_HOST" -u "$DB_USER" -p"$DB_PASS" "$DB_PATH" -e "SELECT id_ultima FROM operaciones_es;" -B | tail -n 1)
    last_id=$(mysql -h "$DB_HOST" -u "$DB_USER" -p"$DB_PASS" "$DB_PATH" -e "SELECT id_ultima FROM operaciones_es;" -B | tail -n 1)
    min_read=$(mysql -h "$DB_HOST" -u "$DB_USER" -p"$DB_PASS" "$DB_PATH" -e "SELECT lecturas_es FROM operaciones_es;" -B | tail -n 1)
    min_write=$(mysql -h "$DB_HOST" -u "$DB_USER" -p"$DB_PASS" "$DB_PATH" -e "SELECT escrituras_es FROM operaciones_es;" -B | tail -n 1)
    operaciones_tot=$(mysql -h "$DB_HOST" -u "$DB_USER" -p"$DB_PASS" "$DB_PATH" -e "SELECT operaciones_tot FROM operaciones_es;" -B | tail -n 1)
fi

#Insertamos en base de datos de mysql
mysql -h "$DB_HOST" -u "$DB_USER" -p"$DB_PASS" "$DB_PATH" -e "INSERT INTO operaciones_es VALUES ($operaciones_es,$lecturas_es,$escrituras_es,$timestamp,$min_read,$min_write,$operaciones_tot);"

sleep 60
done
```

Figura 2.8: Operaciones de E/S

2.9. Estadísticas de red. Bytes enviados y recibidos

Similar al caso de las operaciones de entrada/salida, en este script, tras usar el comando *"ip-slinkshowens18"* obtenemos las estadísticas de red. De estas debemos coger los campos que nos interesan, que corresponden al primer campo de la línea que comienza por *"TX"* (transfer) y el primer campo de la línea que comienza por *"RX"* (receive).

Como estos son datos acumulados, debemos obtener además los datos acumulados de la última entrada, guardados en la tabla *"bytes_red"*, para poder restárselos a los bytes acumulados obtenidos en el comando anterior. Con esto obtendremos los bytes totales, enviados y recibidos desde la última iteración del script.

```
while true;do
  #Obtenemos el timestamp
  timestamp=$(date +%H-%M-%S)
  #Elimino los datos que tengan mas de un día de antigüedad
  mysql -h "$DB_HOST" -u "$DB_USER" -p"$DB_PASS" "$DB_PATH" -e "DELETE FROM bytes_red where col_timestamp < NOW() - INTERVAL 1 DAY;"
  #Obtenemos los bytes enviados y recibido
  bytes_enviados=$(ip -s link show ens18 | awk '/TX/ {print $1}')
  bytes_recibidos=$(ip -s link show ens18 | awk '/RX/ {print $1}')
  last_env=$(mysql -h "$DB_HOST" -u "$DB_USER" -p"$DB_PASS" "$DB_PATH" -e "SELECT bytes_enviados from bytes_red;" -s -N | tail -n 1)
  if [ -z "$last_env" ]; then
    min_env=0
    min_rec=0
  else
    min_env=$((bytes_enviados-last_env))
    last_rec=$(mysql -h "$DB_HOST" -u "$DB_USER" -p"$DB_PASS" "$DB_PATH" -e "SELECT bytes_recibidos from bytes_red;" -s -N | tail -n 1)
    min_rec=$((bytes_recibidos-last_rec))
  fi
  #Completar con datos de la base de datos
  mysql -h "$DB_HOST" -u "$DB_USER" -p"$DB_PASS" "$DB_PATH" -e "INSERT INTO bytes_red VALUES ($bytes_enviados, $bytes_recibidos, $timestamp, $min_env, $min_rec);"
  sleep 10
done
```

Figura 2.9: Estadísticas de red. Bytes enviados y recibidos

2.10. Conexiones de red activas

Lo primero que hacemos en este script es borrar los datos de la tabla para mostrar solo las conexiones actuales.

Para sacar los datos usamos el comando *"ss -tupna"* eliminando la primera linea con *"tail -n + 2"* para eliminar la leyenda de los datos, y con *"awk"* recogemos las lineas que nos interesan, en este caso el protocolo usado, el estado del protocolo, la direccion local y la remota.

Realizamos este proceso en bucle, y por cada iteracion incrementamos la variable indice en una unidad, que hara de clave primaria en la tabla.

```
while true;do
#Obtenemos las conexiones que hubiera anteriormente para guardar las nuevas
mysql -h "$DB_HOST" -u "$DB_USER" -p"$DB_PASS" "$DB_PATH" -e "INSERT INTO conexiones_red"
indice=1
#Obtenemos la informacion de cada uno de las conexiones red
ss -tupna | tail -n + 2 | while read line; do
#Obtenemos para cada uno de los campos por cada uno de los procesos
protocolo=$(echo "$line" | awk '{print $1}')
direccion_lo_local=$(echo "$line" | awk '{print $2}')
direccion_lo_remota=$(echo "$line" | awk '{print $3}')
#Insertamos una fila, añadiendo el campo del pid, para luego actualizar el resto de columnas de la tabla con el resto de valores
mysql -h "$DB_HOST" -u "$DB_USER" -p"$DB_PASS" "$DB_PATH" -e "INSERT INTO conexiones_red (numero_proceso,timestamp) VALUES ($indice,$timestamp)"
mysql -h "$DB_HOST" -u "$DB_USER" -p"$DB_PASS" "$DB_PATH" -e "UPDATE conexiones_red SET protocolo=$protocolo, direccion_lo_local=$direccion_lo_local, direccion_lo_remota=$direccion_lo_remota WHERE numero_proceso=$indice"
indice=$((indice + 1))
done
sleep 10
done
```

Figura 2.10: Conexiones de red activas

2.11. Numero de procesos en ejecucion

Script sencillo que usa el comando *"ps -eo"* que permite ver toda la lista de procesos, solo sacando por pantalla el pid. Este comando se lo debemos pasar al comando *"tail -n + 2"* para que nos omita la parte de la cabecera, que simplemente consiste en informacion y un salto de linea. Tras esto contamos el numero de filas que genera ese comando y lo guardamos en la variable *"num_proceso."*

```
while true;do
#Obtenemos el timestamp
timestamp=$(date +%Y-%m-%d %H:%M:%S)
#Elimino los datos que tengan mas de un dia de antigüedad
mysql -h "$DB_HOST" -u "$DB_USER" -p"$DB_PASS" "$DB_PATH" -e "DELETE FROM nprocesos_ejecuciones where col_timestamp < NOW() - INTERVAL 1 DAY;"
#Obtener el número de procesos en ejecucion
num_procesos=$(ps -eo pid | tail -n + 2 | wc -l)
#Completar con datos de la base de datos
mysql -h "$DB_HOST" -u "$DB_USER" -p"$DB_PASS" "$DB_PATH" -e "INSERT INTO nprocesos_ejecuciones VALUES ($num_procesos, $timestamp);"
sleep 10
done
```

Figura 2.11: Numero de procesos en ejecucion

2.12. Lista detallada de procesos en ejecucion

Este script lo usamos para rellenar 2 tablas de la base de datos simultaneamente. Por un lado, la informacion de los procesos, que incluye su pid, el comando que lo llama, el usuario que lo ejecuta, el estado en el que se encuentra, los porcentajes de cpu y memoria que consumen, la ruta al comando que lo ejecuta, el pid del padre. Y por ultimo una variable que nos ayudara para la otra parte del script.

Para obtener todos estos datos, simplemente debemos usar el comando *"ps"*, que gracias a la ayuda de un bucle while, ira linea por linea leyendo la informacion de cada proceso y con el comando *"awk"*, podremos coger cada trozo de informacion.

En la segunda parte, el script nos servira para crear en grafana un *"nodegraph"* con las relaciones entre los procesos. Esto lo conseguimos con otra tabla, ademas de la anterior, en la que indicaremos cada pid del proceso, junto con el pid del padre y un identificador formado por: *< pid_hijo > - < pid_padre >*. En la tabla anterior, la variable *"noderadius"*, asociada a la cantidad de cpu que consume cada proceso, la usaremos para que en el grafo los nodos se muestren mas grandes o mas pequenos, dependiendo de la cantidad de cpu que consuman. Para obtener los datos para esta tabla, simplemente cogeremos los datos de la tabla anterior, anadiendo el identificador como ya hemos mencionado.

Para las 2 tablas, llamadas *"info_procesos"* y conexiones, debemos tener en cuenta antes de todo que hay que borrarlas antes de empezar una nueva iteracion del script, para que no haya procesos anteriores que se guarden en el sistema/procesos duplicados.

[illegible]

Figura 2.12: Lista detallada de procesos en ejecucion

2.13. Hilos

Lo primero que hace el script es borrar los datos de la base de datos que tengan mas de un dia de antigüedad para evitar consumir mucha memoria.

A continuacion, vamos leyendo cada linea que nos proporciona el comando `"top -H -b -n 1 tail -n +2"`, el cual nos muestra una instantanea del uso de recursos por hilo, excluyendo la linea de cabecera. Dentro del bucle, obtenemos el pid del proceso del que estamos leyendo la informacion, despues el consumo de CPU y memoria por parte de este y el nombre del comando o proceso que esta ejecutando el hilo e insertamos en la base de datos esta informacion.

Repetimos este proceso con cada hilo que se nos muestra en la instantanea.

```
while true;do
#Obtenemos el timestamp
timestamp=$(date +%s-%m-%d %H:%M:%S)
#Eliminamos los datos que tengan mas de un dia de antigüedad
mysql -h "$DB_HOST" -u "$DB_USER" -p"$DB_PASS" --"$DB_PATH" -e "DELETE FROM hilos where cal_timestamp < NOW() - INTERVAL 1 DAY;"
top -b -V -n 1 | tail -n 1 | while read -r linea; do
PID=$(echo $linea | awk '{print $1}');
CPU=$(echo $linea | awk '{print $9}');
MEM=$(echo $linea | awk '{print $10}');
COMMAND=$(echo $linea | awk '{print $12}');
mysql -h "$DB_HOST" -u "$DB_USER" -p"$DB_PASS" --"$DB_PATH" -e "INSERT INTO hilos VALUES ('$PID','$CPU','$MEM','$COMMAND','$timestamp');"
done
sleep 60
done
```

Figura 2.13: Lista detallada de procesos en ejecucion

2.14. Inodos

Lo primero que hacemos es definir las variables para el numero total de inodos usados y el numero total de inodos libres. A continuacion, vamos leyendo cada linea que nos proporciona el comando `"df -i | tail -n +2"`, el cual nos muestra en pantalla el uso de los inodos de los sistemas de archivos mostrados, excluyendo la linea de cabecera.

Dentro del bucle, obtenemos el numero de inodos usados del primer sistema de archivos que nos aparece en el comando anteriormente mencionado utilizando para ello la funcion `"awk"`. Y hacemos la mismo con el numero de inodos libres. Cuando ya tenemos ambas variables, lo siguiente que vamos a hacer es sumar a la cantidad de inodos totales usados y libres el numero de inodos usados y libres respectivamente.

Vamos repitiendo este proceso con cada linea que nos muestra el comando y una vez que hemos terminado, insertamos en la tabla de la base de datos correspondiente los datos que necesitamos guardar en ella.

```
#Obtenemos el timestamp
timestamp=$(date +%Y-%m-%d %H:%M:%S)
total_libres=0
total_usados=0

#Elimino los datos que tengan mas de un día de antigüedad
mysql -h "$DB_HOST" -u "$DB_USER" -p"$DB_PASS" "$DB_PATH" -e "DELETE FROM inodos where col.timestamp < NOW() - INTERVAL 1 DAY;"

#Obtenemos el numero de inodos usados y libres
while read linea; do
    inodos_usados=$(echo $linea | awk '{print $1;}')
    inodos_libres=$(echo $linea | awk '{print $6;}')

    total_usados=$((inodos_usados+total_usados))
    total_libres=$((inodos_libres+total_libres))
done < $(df -i | tail -n +2)
mysql -h "$DB_HOST" -u "$DB_USER" -p"$DB_PASS" "$DB_PATH" -e "INSERT INTO inodos VALUES ($total_usados,$total_libres,$timestamp)"
sleep 60
done
```

Figura 2.14: Lista detallada de procesos en ejecucion

2.15. Interrupciones y forks

En este script, gracias al comando *vmstat -s*, podemos ver ciertas características de nuestro sistema. Entre ellos, se encuentran el número de forks y las interrupciones. Debemos obtener, para cada medida, la línea que contiene los datos requeridos y obtener el campo \$1 con *awk* para obtener el número exacto.

Similar a otros scripts, este dato equivale a valores acumulados desde el comienzo del sistema. Estos valores guardaremos en la base de datos para el uso posterior, pero debemos obtener cada vez el número de forks/interrupciones en el último intervalo de tiempo, restando las acumuladas actuales con las acumuladas de la iteración anterior.

```
Obtenemos la fecha actual
timestamp=$(date '+%Y-%m-%d %H:%M:%S')

#Elimino los datos que tengan mas de un día de antigüedad
mysql -h "$DB_HOST" -u "$DB_USER" -p"$DB_PASS" "$DB_PATH" -e "DELETE FROM interrupciones_forka where col_timestamp < NOW() - INTERVAL 1 DAY;"

num_interrupciones=$(vmstat -s | grep 'interrupts' | awk '{print $1}')
forks=$(vmstat -s | grep 'forks' | awk '{print $1}')

last_forks=$(mysql -h "$DB_HOST" -u "$DB_USER" -p"$DB_PASS" "$DB_PATH" -e "SELECT tot_forks FROM interrupciones_forka" -s -N | tail -n 1)
last_interrupts=$(mysql -h "$DB_HOST" -u "$DB_USER" -p"$DB_PASS" "$DB_PATH" -e "SELECT tot_interrupts FROM interrupciones_forka" -s -N | tail -n 1)
if [ -n "$last_forks" ]; then
    int_forks=$((forks - last_forks))
    int_interrupts=$((num_interrupciones - last_interrupts))
else
    int_interrupts=$((num_interrupciones))
    int_forks=$((forks))
fi

mysql -h "$DB_HOST" -u "$DB_USER" -p"$DB_PASS" "$DB_PATH" -e "INSERT INTO interrupciones_forka (col_timestamp, tot_interrupts, int_interrupts, tot_forks, in
--forks) VALUES ('$timestamp', $num_interrupciones, $int_interrupts, $forks, $int_forks);"

sleep 60
done
```

Figura 2.15: Lista detallada de procesos en ejecución

2.16. Sesiones

El script entra en un bucle infinito (*while true*) diseñado para monitorear continuamente las sesiones de usuario. Al inicio de cada ciclo de este bucle, obtiene el nombre del servidor local usando el comando *hostname*. Una decisión clave en la estrategia es mantener la tabla actualizada solo con las sesiones actualmente activas. Para ello, ejecuta un comando *TRUNCAT ETABLE \$DB_TABLE;* usando *mysql*, eliminando todos los registros existentes antes de buscar nuevas sesiones. Esto asegura que la información refleje fielmente el estado presente.

Luego, el script se enfoca en identificar las sesiones de usuario activas. Utiliza el comando *"who"* para listar los usuarios conectados y lo filtra con *"grep -E 'pts/[0-9]+'"* para seleccionar específicamente sesiones de pseudo-terminales (típicas de conexiones remotas). La salida se procesa línea por línea dentro de un bucle *while read*.

Dentro de este bucle interno, para cada sesión activa, el script extrae el usuario y la *ip_origen* de la conexión. Procede entonces a enriquecer la información intentando geolocalizar la dirección IP de origen, pero aplica una lógica importante: solo intenta geolocalizar IPs públicas. Utiliza una expresión regular para detectar y omitir IPs privadas (RFC 1918) o de loopback. Si la IP no es pública, la latitud y longitud permanecerán NULL.

Si la IP es pública, el script consulta la API de *ipinfo.io* mediante *curl* para obtener las coordenadas geográficas (latitud, longitud). Incluye validaciones para confirmar que la consulta a la API fue exitosa y que los datos de localización recibidos son numéricos y válidos. Solo si todas estas condiciones se cumplen, se asignan los valores de latitud y longitud; de lo contrario, permanecen NULL, asegurando la integridad de los datos geográficos almacenados.

Se añaden los datos a la tabla *registro_sesiones*. Tras procesar todas las sesiones activas, el script hace una pausa de 60 segundos (*sleep 60*) y el bucle *while true* se repite, comenzando de nuevo con el truncado de la tabla y la recopilación actualizada de sesiones. De esta forma, se mantiene la base de datos constantemente al día para su consulta y análisis.

3. Base de datos

En esta seccion se van a detallar todas las tablas usadas en la base de datos del sistema. Cada script guarda los datos en una tabla diferente, por lo que explicaremos cada tabla con sus datos respectivos y el tipo de datos.

3.1. info_carga_cpu

La tabla info_carga_cpu almacenara un id, de tipo entero, la fecha y el valor de la carga de tipo float. La primary key sera el identificador.

Field	Type	Null	Key	Default	Extra
id	int(11)	NO	PRI	NULL	auto_increment
fecha	timestamp	NO		current_timestamp()	
carga_instantanea	float	NO		NULL	

Figura 3.1: info_carga_cpu

3.2. frecuencia_cpu

La tabla frecuencia_cpu almacenara un id, de tipo entero, la velocidad a la que trabaja el procesador(frecuencia), y la fecha. La primary key sera el identificador.

Field	Type	Null	Key	Default	Extra
id	int(11)	NO	PRI	NULL	auto_increment
velocidad	decimal(5,2)	YES		NULL	
col_timestamp	timestamp	NO		current_timestamp()	on update current_timestamp()
fecha	datetime	YES		current_timestamp()	

Figura 3.2: frecuencia_cpu

3.3. uso_ram_swap

La tabla uso_ram_swap almacenara la ram usada de tipo float, la ram libre de tipo float, la swap usada de tipo float, la swap libre de tipo float y la fecha. La primary key sera la fecha.

Field	Type	Null	Key	Default	Extra
usada_ram	float	YES		NULL	
libre_ram	float	YES		NULL	
usada_swap	float	YES		NULL	
libre_swap	float	YES		NULL	
col_timestamp	timestamp	NO	PRI	current_timestamp()	on update current_timestamp()

Figura 3.3: uso_ram_swap

3.4. espacio_disco

La tabla espacio_disco almacenara el espacio usado de tipo float, el espacio libre de tipo double, el nombre de la particion del disco, un identificador de tipo entero y la fecha. La primary key sera el identificador.

Field	Type	Null	Key	Default	Extra
usado	float	YES		NULL	
libre	float	YES		NULL	
nombre	varchar(50)	YES		NULL	
id	int(11)	NO	PRI	NULL	
col_timestamp	timestamp	NO		current_timestamp()	on update current_timestamp()

Figura 3.4: espacio_disco

3.5. operaciones_es

La tabla operaciones_es almacenara las operaciones por segundo de tipo int, los kb leidos de tipo int, los kb escritos de tipo int, la fecha, los kb leidos en un minuto de tipo int, los kb escritos en un minuto de tipo int y las operaciones totales de tipo int. La primary key sera la fecha.

Field	Type	Null	Key	Default	Extra
operaciones_segundo	int(11)	YES		NULL	
kb_leidos	int(11)	YES		NULL	
kb_escritos	int(11)	YES		NULL	
col_timestamp	timestamp	NO	PRI	current_timestamp()	on update current_timestamp()
kb_leidos1min	int(11)	YES		NULL	
kb_escritos1min	int(11)	YES		NULL	
operaciones_tot	int(11)	YES		NULL	

Figura 3.5: operaciones_es

3.6. bytes_red

La tabla bytes_red almacenara los bytes enviados de tipo int, los bytes recibidos de tipo mediumtext, la fecha, los bytes enviados anteriormente de tipo int y los bytes recibidos anteriormente. La primary key sera la fecha.

Field	Type	Null	Key	Default	Extra
bytes_enviados	int(11)	YES		NULL	
bytes_recibidos	mediumtext	YES		NULL	
col_timestamp	timestamp	NO	PRI	current_timestamp()	on update current_timestamp()
min_env	int(11)	YES		NULL	
min_rec	int(11)	YES		NULL	

Figura 3.6: bytes_red

3.7. conexiones_red

La tabla conexiones_red almacenara el numero de conexion del proceso actual de tipo int, el protocolo que usa, el estado, la direccion ip local y la direccion ip remota. La primary key sera el numero de conexion.

Field	Type	Null	Key	Default	Extra
numero_conexion	int(11)	NO	PRI	NULL	
protocolo	varchar(10)	YES		NULL	
estado	varchar(20)	YES		NULL	
direccion_ip_local	varchar(50)	YES		NULL	
direccion_ip_remota	varchar(50)	YES		NULL	

Figura 3.7: conexiones_red

3.8. nprocesos_ejecuciones

La tabla nprocesos_ejecuciones almacenara el numero de procesos en ejecucion de tipo decimal con un maximo de 30 digitos enteros y la fecha. La primary key sera la fecha.

Field	Type	Null	Key	Default	Extra
numero_procesos	decimal(30,0)	YES		NULL	
col_timestamp	timestamp	NO	PRI	current_timestamp()	on update current_timestamp()

Figura 3.8: nprocesos_ejecuciones

3.9. info_procesos

La tabla info_procesos almacenara el pid del proceso actual de tipo entero, el comando, el usuario, el estado del proceso, el porcentaje de cpu que usa de tipo decimal con cinco enteros y dos decimales, el porcentaje de memoria que usa con cinco enteros y dos decimales, la ruta del comando, el pid del proceso padre de tipo entero y el tamano del proceso de tipo entero. La primary key sera el pid del proceso actual.

Field	Type	Null	Key	Default	Extra
pid	int(11)	NO	PRI	NULL	
comando	varchar(100)	YES		NULL	
usuario	varchar(30)	YES		NULL	
estado	varchar(30)	YES		NULL	
cpu_porcentaje	decimal(5,2)	YES		NULL	
mem_porcentaje	decimal(5,2)	YES		NULL	
ruta_comando	varchar(500)	YES		NULL	
pid_father	int(11)	YES		NULL	
tamano	int(11)	YES		NULL	

Figura 3.9: info_procesos

4. Graficas y Valores

En esta seccion se adjuntaran ejemplos de los valores obtenidos de rendimiento del servidor.

4.1. ESPACIO USADO Y LIBRE EN DISCO

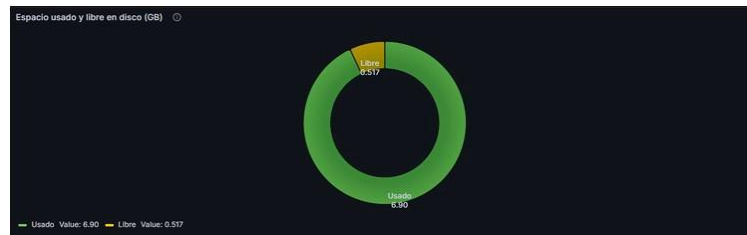
Relacion de la metrica con el rendimiento de un sistema informatico:

Esta metrica es importante ya que la gestion del disco es un aspecto fundamental para mantener un buen rendimiento del sistema informatico. Un disco duro o casi lleno puede llegar a provocar una lentitud, errores y otros problemas que afectan negativamente a la productividad o a la experiencia del usuario.

Seleccion del tipo de grafica:

Para representar este tipo de metrica en Grafana hemos utilizado un grafico de tipo Pie Chart o mas comunmente conocido como diagrama de sectores, ya que consideramos que lo mas importante en esta metrica es representar la proporcion de espacio usado y libre con respecto al total del disco en un instante de tiempo determinado y no como evoluciona el mismo a lo largo del tiempo.

Asi mismo, la existencia de dos colores para diferenciar el espacio usado del espacio libre y viceversa junto con la facilidad que este grafico presenta para comparar el tamano de los segmentos, nos permite rapidamente observar si el disco esta mas lleno o vacio o detectar algun problema potencial en el caso de que el segmento de espacio usado sea mucho mayor que el de espacio libre.



4.2. USO DE RAM Y SWAP

Relacion de la metrica con el rendimiento de un sistema informatico:

Esta metrica es importante ya que nos permite monitorizar el sistema informatico a analizar. La memoria RAM nos proporciona un acceso rapido a los datos mientras que el SWAP actua como una extension de esta, de manera que si nosotros mediante esta metrica observamos que la memoria RAM esta constantemente llena y que el sistema esta utilizando mucho el SWAP, podemos intuir un mal rendimiento del sistema, con aplicaciones lentas o cuelgues. Por lo tanto, nos permite observar como esta funcionando el sistema en terminos de memoria y evaluar el posible rendimiento de este con esta informacion.

Seleccion del tipo de grafica:

Para representar este tipo de metrica en Grafana hemos utilizado un grafico de tipo Bar chart. Se ha elegido este tipo de grafico ya que nos permite mezclar dos caracteristicas:

La primera es que nos permite mostrar la cantidad absoluta de memoria libre y usada y comparar visualmente, el uso de la RAM con el de la SWAP, indicando asi si se esta recurriendo excesivamente al uso de la SWAP (barra de SWAP muy alta en comparacion con la barra de la RAM).

Por otra parte, este grafico nos permite obtener un historial del uso de ambas memorias en mi sistema con lo que podemos detectar tendencias, patrones, anomalias que hagan que el rendimiento del sistema se ve afectado, deteccion de picos que indiquen problemas puntuales, identificar momentos del dia en el que el sistema se encuentra bajo una mayor carga de memoria permitiendonos planificar actividades que consumen muchos recursos fuera de este horario,etc...



4.3. DIFERENCIA DE INODOS DEL DISCO DURO

Relacion de la metrica con el rendimiento de un sistema informatico:

Esta metrica es importante ya que los inodos son una estructura de datos que nos permite almacenar informacion sobre un archivo o directoria, cada uno de los cuales tiene asociado un inodo del sistema. El numero de inodos es fijo al crear el sistema de archivos, por lo que el sistema solo puede almacenar una cantidad limitada de archivos independientemente del espacio libre en disco, si los inodos se agotan.

Por una parte, nos permite observar si se esta produciendo un aumento rapido y significativo en el numero de inodos utilizados lo que puede ser una senal de que el sistema se esta acercando al limite de inodos produciendo asi un agotamiento de estos que llevaria a un estado en el que el sistema es practicamente inutilizable al no poder crear nuevos archivos y

directorios.

Selección del tipo de gráfica:

Para representar este tipo de métrica en Grafana hemos utilizado un gráfico de tipo Time series. Lo importante de esta métrica no es mostrar el valor puntual de la diferencia de inodos en un momento dado, sino la tendencia a lo largo del tiempo para observar a qué ritmo está aumentando el número de inodos, si hay patrones recurrentes, etc... También nos facilita la identificación de picos, anomalías, cambios bruscos, etc... que nos sirve como señal de un posible problema (ataque DoS o un error en una aplicación)

Por último, al representar esta métrica de esta forma podemos combinar este gráfico con otros como pueden ser el uso de CPU, el uso de disco para poder correlacionar eventos y determinar si ciertos patrones en una métrica causan cambios en otras.



4.4. USO DE CPU Y MEMORIA POR PARTE DE LOS HILOS

Relación de la métrica con el rendimiento de un sistema informático:

Esta métrica es importante ya que los hilos son la unidad más pequeña que puede ser gestionada de forma independiente por parte del sistema operativo. Un alto uso de CPU por parte de estos

supone que el sistema esta trabajando de intensamente, con lo que podremos observar si este alto uso es continuado detectando asi una posible sobrecarga o cuello de botella en el procesamiento.

Por otra parte, el uso de memoria indica cuanta RAM se esta consumiendo en el sistema, si esta memoria se agota el sistema comenzara a utilizar la memoria virtual, tambien conocida como SWAP, que es mucho mas lenta, degradando asi el rendimiento del sistema informatico.

Combinando ambas metricas podemos determinar si el sistema se encuentra limitado por la potencia de procesamiento (CPU), en caso de que la CPU este al maximo y la memoria baja o limitado por la memoria en el caso contrario.

Por ultimo, podemos observar tendencias que indiquen cargas intensivas de trabajo o picos repentinos que nos puedan advertir de algun ataque malicioso.

Selección del tipo de grafica:

Para representar este tipo de metrica en Grafana hemos utilizado un grafico de tipo Time series. Lo importante de esta metrica no es mostrar el valor puntual del consumo de CPU o memoria por parte de los hilos en un momento dado, sino la tendencia a lo largo del tiempo para observar a que ritmo esta aumentando/reduciendo ambos consumos, si hay patrones recurrentes,etc...

Tambien nos facilita la identificación de picos, anomalías, cambios bruscos,etc... que como hemos mencionado anteriormente, nos sirven para detectar posibles ataques maliciosos, determinar las horas de mayores cargas del sistema o ver como afectan algunos cambios en otras metricas o en el sistema a esta.

Por ultimo, al representar esta metrica de esta forma podemos construir unos modelos de prediccion que pronostiquen el comportamiento futuro de esta metrica y con este grafico es facil superponer unos umbrales, en forma de lineas horizontales, que nos permiten identificar cuando estas metricas exceden estos umbrales.



4.5. NUMERO DE PROCESOS

Relacion de la metrica con el rendimiento de un sistema informatico:

El numero total de procesos en un sistema operativo es una metrica fundamental, que afecta directamente al rendimiento general. Cada proceso activo o en espera consume recursos del sistema, como por ejemplo:

Memoria (RAM): Cada proceso requiere un espacio de direccionamiento virtual y consume memoria fisica. Si la demanda excede la RAM fisica disponible, el sistema recurrira a mecanismos de paginacion, siendo considerablemente mas lento que el acceso a memoria RAM.

Tiempo de CPU: Un numero elevado de procesos listos para ejecutarse incrementa la carga sobre el planificador de procesos y puede aumentar la frecuencia de los cambios de contexto, que introduce sobrecargas.

Otros Recursos: Los procesos tambien consumen otros recursos finitos, como descriptores de archivo, identificadores de proceso (PIDs) y estructuras de datos del kernel.

Seleccion del tipo de grafica:

Para representar estos datos hemos elegido el grafico de lineas. Este tipo de grafico permite observar de manera efectiva las tendencias, la estabilidad o las fluctuaciones en el numero total de procesos existentes en el sistema a lo largo del periodo de monitorizacion. Facilita la identificacion de patrones, incrementos, decrementos o periodos de estabilidad en la carga de procesos.



4.6. INTERCONEXION DE PROCESOS

Relacion de la metrica con el rendimiento de un sistema informatico:

Si bien no cuantifica directamente una métrica de rendimiento como el uso de CPU o memoria, la visualización del árbol de procesos es fundamental para el análisis y la depuración del rendimiento. Permite:

Identificación de Orígenes: Rastrear procesos que consumen recursos excesivos hasta su proceso padre, ayudando a identificar la aplicación o servicio responsable original.

Análisis de Dependencias: Comprender cómo se inician y gestionan los servicios y aplicaciones en el sistema, lo cual puede ser relevante si un proceso padre finaliza inesperadamente, afectando a sus hijos.

Diagnóstico de Problemas: Detectar estructuras anómalas, como una proliferación inesperada de procesos hijos bajo un mismo padre, que podría indicar un error de software o una carga inusual.

Selección del tipo de gráfica:

Para representar estos datos hemos elegido hacer un grafo. Este tipo de visualización es óptimo para representar relaciones jerárquicas o de dependencia entre entidades. En este contexto, los nodos representan los procesos individuales y los edges, ilustran la relación padre-hijo, obtenida a través del PID y el ParentPID. El grafo resultante permite visualizar la estructura del árbol de procesos del sistema, mostrando que procesos han sido iniciados por otros.



4.7. INFORMACION DE PROCESOS

Relacion de la metrica con el rendimiento de un sistema informatico:

Consumo de Recursos: Las columnas de %cpu y %mem, cuantifican directamente la utilizacion de estos recursos criticos por cada proceso, permitiendo identificar cuales estan consumiendo mas y potencialmente causando ralentizaciones.

Estado del Proceso: La columna 'estado', ofrece informacion vital sobre la actividad del proceso. Un alto numero de procesos en estado 'R' puede indicar contencion de CPU, mientras que procesos en 'D' suelen senalar esperas de E/S (disco, red), y procesos 'Z' indican problemas que requieren atencion.

Atribucion: La informacion de 'usuario' y 'comando' permite atribuir el consumo de recursos a usuarios o aplicaciones especificas, facilitando la gestion y optimizacion.

Seleccion del tipo de grafica:

Para representar estos datos hemos elegido hacer una tabla. La visualizacion en formato de tabla es la mas adecuada para presentar informacion detallada de cada uno de los procesos. Cada fila representa un proceso unico, mientras que las columnas muestran metricas especificas asociadas a ese proceso. Este formato facilita la consulta de los distintos valores y su visualizacion conjunta permite la comparacion entre las metricas.

Info. procesos ⓘ				
pid	comando	usuario	estado	cp
16659	grafana	grafana	S	
30736	sshd	root	S	
30440	sshd	usuario	S	
682	qemu-ga	root	S	
31048	bash	usuario	R	
31047	ps	usuario	R	

4.8. INTERRUPCIONES Y FORKS

Relacion de la metrica con el rendimiento de un sistema informatico:

Interrupciones: Son senales que demandan atencion inmediata de la CPU, interrumpiendo la tarea en curso. El manejo de cada interrupcion consume ciclos de CPU (overhead). Un exceso de

interrupciones reduce el tiempo de CPU disponible para las aplicaciones y aumenta los cambios de contexto, pudiendo degradar la capacidad de respuesta y el rendimiento general.

Forks: La llamada al sistema `fork()` es el mecanismo primario para crear nuevos procesos en sistemas Unix/Linux. Cada fork implica la duplicación del espacio de direcciones del proceso padre y la asignación de recursos del kernel, consumiendo CPU y memoria. Una tasa alta de forks puede ser un síntoma de problemas como bucles descontrolados que crean procesos, agotando rápidamente los recursos y llevando a la inestabilidad del sistema.

Selección del tipo de gráfica:

Para representar estos datos hemos elegido un gráfico de barras por intervalos. La visualización empleada, que muestra barras para representar la cantidad de eventos (interrupciones y forks) ocurridos dentro de intervalos de tiempo discretos (cada minuto, según el cálculo `int_interrupts` e `int_forks` del script), es adecuada para comparar la tasa o frecuencia de estos eventos a lo largo del tiempo. Permite identificar rápidamente los periodos con mayor o menor actividad de interrupciones y creación de procesos, facilitando la correlación visual entre ambas métricas y la detección de picos de actividad.



4.9. OPERACIONES ENTRADAS/SALIDAS

Relacion de la metrica con el rendimiento de un sistema informatico:

Tasa de Transferencia (KB Leidos/Escritos por Minuto): Mide el throughput de datos hacia y desde los dispositivos de almacenamiento. Una tasa elevada sostenida puede indicar una carga de E/S intensiva. Si esta tasa se aproxima o excede la capacidad maxima del hardware, la latencia de las operaciones aumentara en gran medida.

Tasa de Operaciones (Operaciones Totales por Minuto): Mide el numero de solicitudes de lectura y escritura individuales. Es especialmente relevante para cargas de trabajo con acceso aleatorio o a muchos ficheros pequenos (bases de datos, servidores virtuales). Los dispositivos de almacenamiento tienen un limite de IOPS que pueden manejar. Exceder este limite causa alta latencia, incluso si la tasa de transferencia en KB/min no es maxima.

Selección del tipo de grafica:

Para representar estos datos hemos elegido un grafico de barras agrupadas. Se utiliza un grafico de barras agrupadas para representar las distintas facetas de la actividad de entrada/salida (E/S) por intervalo de tiempo. Este formato permite comparar visualmente y de forma simultanea las magnitudes de KB de las operaciones



4.10. SESIONES

Relacion de la metrica con el rendimiento de un sistema informatico:

Se detecta que entidad, ya sea un usuario SSH, procesos que operan en secreto o software malicioso, esta absorbiendo los recursos del sistema, lo cual afecta la memoria y el uso del procesador. Un pico inesperado en el consumo puede indicar intrusiones, como ataques de fuerza bruta, o programas no permitidos, lo que empeora el desempeno del sistema. En ambientes donde varios usuarios comparten recursos, se previene que alguno los acapare. Vigilar esto es esencial para mantener la estabilidad y la seguridad, sobre todo en servidores y equipos compartidos.

Utilidad de la medida de rendimiento:

Detectar accesos inusuales en zonas no esperadas (ejemplo: una conexion en africa si la operacion se centra en Europa).

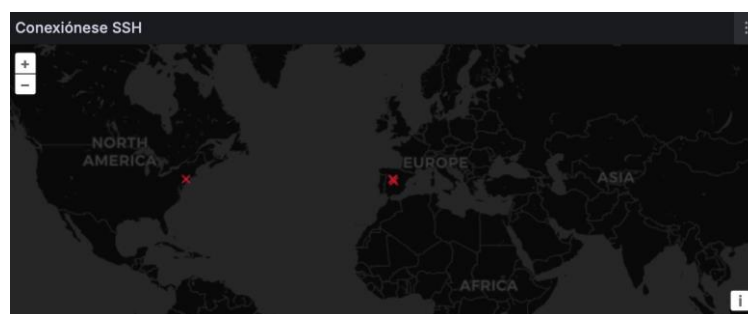
Correlacionar eventos tecnicos con ubicaciones fisicas (picos de carga en servidores regionales).

Validar la cobertura esperada del servicio o infraestructura.

El enfoque visual simplifica el analisis espacial, ideal para equipos tecnicos y de seguridad que requieren precision geografica junto a datos operativos.

Selección del tipo de gráfica:

Hemos seleccionado este mapa de conexiones para visualizar de forma intuitiva la distribución geográfica de las conexiones activas. Las X rojas destacan las ubicaciones clave, permitiendo identificar rápidamente concentraciones de actividad o regiones con mayor tráfico. Al pasar el cursor sobre los marcadores, se accede a detalles técnicos esenciales (usuario, IP, fecha), lo que combina una vista global con información específica sin saturar la pantalla.



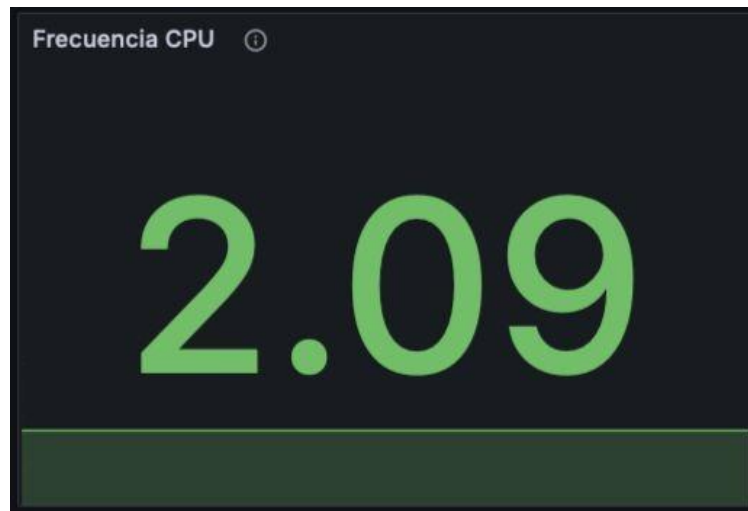
4.11. FRECUENCIA CPU

Relación de la métrica con el rendimiento de un sistema informático:

Observa que tan rápido trabaja el procesador, demostrando su posibilidad para ejecutar tareas pesadas (operaciones matemáticas avanzadas...). Unos valores elevados agilizan su desempeño, aunque mantenerse así por mucho tiempo podría señalar un recalentamiento o un gasto de energía muy alto. Métodos como el turbo boost varían este valor al instante, encontrando un punto medio entre fuerza y temperatura. Resulta primordial para detectar si el equipo está trabajando demasiado o si gasta mucha corriente.

Selección del tipo de gráfica:

Para mostrar la frecuencia hemos decidido mostrar los dígitos. La frecuencia, al ser un valor de tipo doble, sin más datos que acompañan, lo más correcto es mostrar la cifra, pues es lo más limpio y eficiente posible.



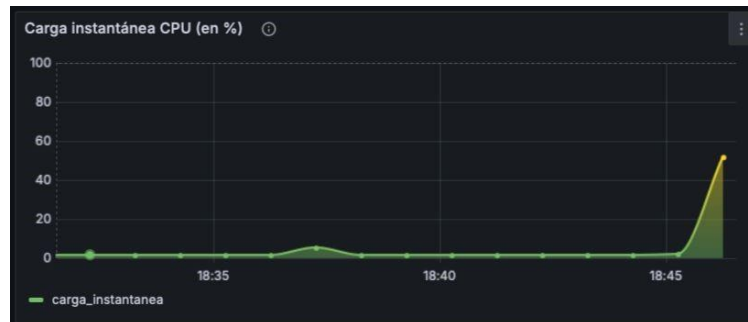
4.12. CARGA CPU

Relación de la métrica con el rendimiento de un sistema informático:

Es un indicador de rendimiento esencial, ya que muestra el volumen de trabajo que el procesador está gestionando en el instante. Unos valores elevados señalan una sobrecarga (más procesos que núcleos operativos), lo que provoca demoras en tareas primordiales, como los servicios web o las bases de datos. Su supervisión posibilita optimizar los recursos, repartir las cargas y prevenir caídas en servidores o máquinas virtuales. Analizarla es crucial para asegurar la eficiencia en contextos de mucha exigencia.

Selección del tipo de gráfica:

Para representar este tipo de métrica en Grafana hemos utilizado un gráfico de tipo Time series. Esta gráfica nos sirve para ver la progresión de la carga en el tiempo, lo cual nos es útil a la hora de detectar picos o crecimientos inesperados en el sistema, a la hora por ejemplo de mapear la eficiencia de un proceso.



4.13. BYTES ENVIADOS Y RECIBIDOS

Relación de la métrica con el rendimiento de un sistema informático:

El rendimiento de un sistema informático está estrechamente relacionado con esta métrica, ya que la velocidad y el volumen de transferencia de datos pueden afectar el funcionamiento general del sistema de varias maneras:

Ancho de Banda y Velocidad de Transferencia. Si el sistema está enviando o recibiendo muchos datos, un mayor ancho de banda puede mejorar el rendimiento al evitar cuellos de botella o retrasos.

Procesamiento de Datos. El sistema necesita procesar y almacenar los datos que envia y recibe. Si se reciben grandes cantidades de datos que deben ser procesadas por el procesador, la memoria o el disco, puede haber latencia y retrasos en la ejecucion de otras tareas. El rendimiento del sistema en terminos de CPU y memoria RAM se ve afectado cuando hay un volumen excesivo de datos que manejar.

Capacidad de los Dispositivos de Almacenamiento. Los bytes enviados y recibidos tambien estan relacionados con el acceso a los dispositivos de almacenamiento. Por ejemplo, en un sistema de bases de datos, si hay muchas solicitudes de lectura y escritura de datos, la velocidad de transferencia de bytes entre la memoria RAM y el disco duro o la SSD afecta directamente el rendimiento. Un sistema con un disco duro lento.

Seleccion del tipo de grafica:

Para representar este tipo de metrica en Grafana hemos utilizado un grafico de tipo Barchart. Se ha elegido este tipo de grafico ya que nos permite mostrar la cantidad de bytes enviados y recibidos y comparar visualmente, detectando si hay algun cuello de botella, o si no se corresponden los bytes con los procesos activos.



4.14. INFORMACION DE CONEXIONES DE RED

Relacion de la metrica con el rendimiento de un sistema informatico:

La informacion de las conexiones de red es importante a la hora de evaluar el rendimiento de un sistema informatico, pues se puede detectar conexiones inesperadas en el sistema, lo cual puede no ser lo mas conveniente a nivel de seguridad. Seleccion del tipo de grafica:

Para mostrar los datos hemos seleccionado una tabla, pues es la manera mas adecuada y ordenada de mostrar todos los datos de una conexion red. Cada conexion tiene varios campos, por lo que de esta manera se ordenan todos y podemos acudir rapidamente a uno de ellos, por ejemplo la ip de un proceso, sin tener que revisar datos apenas.

Conexiones Red Activas ⓘ			
numero_conexion	direccion_ip_local	direccion_ip_remota	estado
1	127.0.0.53%lo:53	0.0.0.0:*	UNCONN
2	0.0.0.0:68	0.0.0.0:*	UNCONN
3	127.0.0.1:3306	0.0.0.0:*	LISTEN
4	0.0.0.0:22	0.0.0.0:*	LISTEN
5	127.0.0.53%lo:53	0.0.0.0:*	LISTEN
6	127.0.0.1:3306	127.0.0.1:49752	ESTAB
7	10.0.31.18:45126	34.117.59.81:443	TIME-WAIT
8	127.0.0.1:3306	127.0.0.1:49742	ESTAB

5. Conclusiones

Durante todo el informe se han expuesto diversos datos, y formas de interpretarlos y representarlos. La conclusion mas evidente es que la informatica no es un ambito facil.

Para resumir la practica, podemos apreciar que un sistema tiene gran cantidad de datos, y que hay que saber manejarlos, pues pueden ser de gran utilidad bien interpretados, pero por contra mal interpretados pueden jugar malas pasadas.

Lo mismo para su representacion, hay que tener un buen criterio y seguir orden para poder acudir a estos datos de manera rapida y eficiente, porque puede haber en juego dinero o la propia integridad del sistema.